REDROB HACKATHON

Intelligent Candidate Discovery & Ranking

Complete Solution Blueprint

Problem Statement . Dataset Analysis . Architecture
Scoring Logic . Tech Stack . Implementation Plan
Submission Guide . Team Roles . Roadmap

Track 01 — The Data & AI Challenge

Prepared for Team Discussion & Implementation Kickoff

Table of Contents

1. Problem Statement Overview

The Redrob Hackathon Track 01 (Data & AI Challenge) asks teams to build an **Intelligent Candidate Ranking System**. The core challenge: rank a pool of 100,000 synthetic job candidates against a single job description and deliver a precise top-100 shortlist, ranked best-fit first. The system must go far beyond keyword matching and demonstrate deep contextual understanding of both the job requirements and each candidate.

> The sample submission provided by the organizers ranks an HR Manager at #1. That is the deliberately **bad** example. A system that just counts AI keyword matches will fail at every evaluation stage. The challenge is explicitly designed to penalise keyword-stuffing and reward contextual, semantic reasoning.

What Makes This Hard

- **Semantic gap.** A candidate who 'built a recommendation system at a product company' may not use the words 'RAG' or 'Pinecone' anywhere. A naive keyword ranker misses them entirely.
- **Keyword stuffers.** Many candidates list every AI buzzword as a skill but have completely unrelated job titles — Marketing Manager, HR Manager. The system must detect and demote these.
- **Honeypot profiles (~80 candidates).** Subtly impossible profiles: 8 years experience at a company founded 3 years ago; 'expert' in 10 skills with 0 months used. Ranking more than 10 of these in your top-100 means instant disqualification at Stage 3.
- **Availability signals.** A perfect-on-paper candidate who last logged in 6 months ago with a 5% recruiter response rate is not actually hirable. Behavioral signals must modify scores.
- **Strict compute budget.** The ranking step must complete under 5 minutes on CPU with no network and 16 GB RAM. No per-candidate LLM API calls. Must use precomputed data + local models.

2. Dataset File-by-File Breakdown

File	Size	What It Is
`candidates.jsonl`	465 MB	100,000 candidate profiles, one JSON object per line. The main dataset your ranker processes.
`sample_candidates.json`	294 KB	First 50 candidates, pretty-printed. Use this during development before touching the full file.
`job_description.docx`	40 KB	The single JD you rank all candidates against. Senior AI Engineer, Redrob AI, Pune/Noida.
`candidate_schema.json`	8.7 KB	JSON Schema defining every field. Reference when building the feature extractor.
`redrob_signals_doc.docx`	37 KB	Explains all 23 behavioral signals in redrob_signals. Critical reading.
`submission_spec.docx`	42 KB	Complete rules: file format, compute constraints, evaluation stages, scoring metrics, honeypot rules.
`validate_submission.py`	5 KB	Run this on your CSV before submitting. Catches all format errors automatically.
`sample_submission.csv`	9 KB	Format reference ONLY — not a quality ranking. Shows the required CSV structure.

File	Size	What It Is
`submission_metadata_template.yaml`	5 KB	Fill this in and commit it to your GitHub repo root. Required for submission.

Note: The README refers to the dataset as 'candidates.jsonl.gz' (~52 MB compressed). In the actual bundle provided, it is already decompressed as candidates.jsonl (465 MB). Load it directly and stream it line-by-line; there is no need to gunzip it.

3. The Job Description — What We Are Ranking For

Role Summary

Field	Value
Title	Senior AI Engineer — Founding Team
Company	Redrob AI (Series A, AI-native talent intelligence platform)
Location	Pune / Noida, India (Hybrid) — open to relocation from Tier-1 cities
Experience	5-9 years applied ML at product companies, NOT services firms
Employment	Full-time

Hard Requirements — Must-Have Skills

✓ Production experience with **embeddings-based retrieval systems** (sentence-transformers, OpenAI embeddings, BGE, E5, or similar). They care about operational experience — embedding drift, index refresh, retrieval-quality regression in production — not just which model.

✓ Production experience with **vector databases or hybrid search infrastructure** — Pinecone, Weaviate, Qdrant, Milvus, OpenSearch, Elasticsearch, FAISS, or similar.

✓ **Strong Python** — code quality matters.

✓ Hands-on experience designing **evaluation frameworks for ranking systems** — NDCG, MRR, MAP, A/B testing, offline-to-online correlation.

Nice-to-Have Skills

- LLM fine-tuning (LoRA, QLoRA, PEFT)
- Learning-to-rank models (XGBoost-based or neural)
- Prior exposure to HR-tech, recruiting, or marketplace products
- Background in distributed systems or large-scale inference
- Open-source AI/ML contributions

Explicit Disqualifiers — The JD's Own Hard No's

These are not inferences — the JD explicitly names these as dealbreakers. Your scoring system must penalise candidates matching these patterns:

Pattern	Why Disqualified	How to Detect in Data
Pure research / academic, no production deployment	Tried twice, bad fit for both sides	`career_history: only academia or research orgs; no product company anywhere`
Entire career at TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini	Consulting-only, no product experience	`All career_history companies are large IT-services firms`
CV / Speech / Robotics specialist, no NLP/IR exposure	Would re-learn fundamentals here	`Skills dominated by image/speech; zero retrieval or NLP signal`

Pattern	Why Disqualified	How to Detect in Data
Primary AI experience is recent (<12 mo) LangChain / framework tutorials	Framework enthusiast, not systems thinker	`Low duration_months on core ML skills; only recent certifications`
Title-chaser: company-hop every 1.5 years for title bumps	Needs a 3+ year commitment	`Many short stints (<18 months) with a title-escalation pattern`
Senior engineer not writing code in last 18 months (pure 'architect')	This role writes code	`Title contains Architect/Principal with no recent hands-on work described`

The JD's own hint to hackathon participants: the right answer is NOT finding candidates whose skills section contains the most AI keywords. The right answer is reasoning about the gap between what the JD says and what it means. A candidate who 'built a recommendation system at a product company' is a fit even without the word 'RAG' anywhere. Read the career_history description fields — that is where the real signal lives.

Ideal Candidate Profile — JD's Own Description

- 6-8 years total experience, of which 4-5 are in applied ML/AI roles at **product companies**
- Shipped at least one end-to-end ranking, search, or recommendation system to real users at scale
- Strong opinions about retrieval (hybrid vs dense), evaluation (offline vs online), and LLM integration (fine-tune vs prompt) — backed by systems they actually built
- Located in or willing to relocate to Noida or Pune
- Active on the Redrob platform, or clear signal of being in the job market

The JD adds: **"We'd rather see 10 great matches than 1000 maybes."** This is a precision-first problem. Combined with NDCG@10 being 50% of the score, the lesson is clear — perfect the top of your list rather than padding the bottom.

4. Candidate Data Schema — Every Field

Each candidate in candidates.jsonl is a JSON object with the keys below. Understanding every field is essential for building the feature extractor.

4.1 profile (object)

Field	Type	Notes and Signal Value
`anonymized_name`	string	Synthetic name. Not useful for ranking.
`headline`	string	One-line self-described role. Soft signal — embed together with summary.
`summary`	string	Multi-sentence free text. HIGH VALUE — embed this. Candidates describe what they have built here.
`location`	string	City, state. JD wants Pune or Noida (or willing_to_relocate = true).
`country`	string	India preferred. JD also accepts Hyderabad, Mumbai, Delhi NCR.
`years_of_experience`	float	Self-reported YoE. JD wants 5-9. Cross-validate against career_history.
`current_title`	string	Current job title. HIGH VALUE — a Marketing Manager is not a fit regardless of skills.
`current_company`	string	Current employer. Classify as product vs consulting vs academia.
`current_company_size`	enum	1-10 / 11-50 / ... / 10001+. Very large often = enterprise or consulting.
`current_industry`	string	Industry of current employer. 'IT Services' is a weak fit signal.

4.2 career_history (array of up to 10 job objects)

The richest part of the profile. Each entry represents one job role:

Field	Type	Notes and Signal Value
`company`	string	Employer name. Classify as product vs consulting vs academia.
`title`	string	Job title at this role. Track progression — does it show ML/AI growth?
`start_date / end_date`	date	Use to verify tenure and detect overlapping or inflated timelines.
`duration_months`	int	Pre-computed tenure. Use for quick tenure and title-chaser checks.
`is_current`	bool	Whether this is their active role.
`industry`	string	Industry of the employer at this role.
`company_size`	enum	Same enum as profile.current_company_size.
`description`	string	FREE TEXT of what they actually did. EXTREMELY HIGH VALUE — embed this. Look for: ranking, retrieval, vector search, recommendation, production ML, A/B testing, embedding drift, NDCG, serving.

4.3 education (array)

Field	Type	Notes and Signal Value
`institution`	string	University name.
`degree`	string	B.Tech, M.Tech, PhD, etc.
`field_of_study`	string	CS, ECE, Stats, Math all good. Non-STEM is a weak negative signal.
`tier`	enum	tier_1 / tier_2 / tier_3 / tier_4 / unknown. Minor positive signal; not a disqualifier.

4.4 skills (array)

Field	Type	Notes and Signal Value
`name`	string	Skill name. Match against JD required and preferred skills list.
`proficiency`	enum	beginner / intermediate / advanced / expert.
`endorsements`	int	Peer endorsements. High count = social validation of the skill.
`duration_months`	int	CRITICAL. Months they have actually USED this skill. An 'expert' with 0 duration_months is a keyword stuffer; an expert with 60+ months is highly credible. The single best signal for detecting fake skills.

4.5 certifications and languages

certifications has name, issuer, year. Relevant certs include AWS ML Specialty, GCP Professional ML Engineer, and the Deep Learning Specialization; recent years are more meaningful. languages is mostly informational — English proficiency is assumed.

5. The 23 Behavioral Signals

Every candidate has a redrob_signals object with 23 platform-activity signals. These are often more predictive of actual hirability than the static profile, and act as a multiplier on top of the skill-match score.

#	Signal	Range	What It Measures	How to Use in Scoring
1	`profile_completeness_score`	0-100	% of profile filled in	Minor positive. Below 50 = suspect profile.
2	`signup_date`	date	When they joined Redrob	Old signup + no recent activity = stale.
3	`last_active_date`	date	Last login date	CRITICAL. Days since active = recency modifier. >180d = heavy penalty.
4	`open_to_work_flag`	bool	Marked available	Strong positive if True. Passive is fine but lower weight.
5	`profile_views_received_30d`	int	Recruiter views last 30d	High = platform-validated interest. Secondary signal.
6	`applications_submitted_30d`	int	Roles applied to recently	High = active job seeker. Positive.
7	`recruiter_response_rate`	0-1	Fraction of messages replied to	CRITICAL. <0.2 = hard to reach. Down-weight.
8	`avg_response_time_hours`	float	Median reply time	Lower = better. >72h = slow to engage.
9	`skill_assessment_scores`	dict	Per-skill Redrob test scores (0-100)	GROUND TRUTH on skills. Expert claim + score <40 = lying.
10	`connection_count`	int	Redrob connections	Weak social signal. Very low = new or inactive.
11	`endorsements_received`	int	Total skill endorsements	Social validation. High = credible.
12	`notice_period_days`	0-180	Stated notice period	JD wants <30d. >90d = penalty. Can buy out 30d.
13	`expected_salary_range_inr_lpa`	min/max	Salary expectations (LPA)	Budget-fit check. Extremely high = risk.
14	`preferred_work_mode`	enum	remote / hybrid / onsite / flexible	JD is hybrid. Remote-only = minor penalty.
15	`willing_to_relocate`	bool	Will relocate if needed	Key for non-Pune/Noida India candidates.
16	`github_activity_score`	-1 to 100	GitHub contribution score	-1 = no GitHub. >60 = active coder. JD values external evidence.
17	`search_appearance_30d`	int	Times shown in searches	Platform-validated relevance signal.
18	`saved_by_recruiters_30d`	int	Recruiters who bookmarked them	Strong positive. Others already found them valuable.

#	Signal	Range	What It Measures	How to Use in Scoring
19	`interview_completion_rate`	0-1	Interviews actually attended	Low = ghosting risk. <0.5 = red flag.
20	`offer_acceptance_rate`	-1 to 1	Historical offer acceptance	-1 = no data. High = reliable. Low = finicky.
21	`verified_email`	bool	Email verified	Basic trust signal.
22	`verified_phone`	bool	Phone verified	Basic trust signal.
23	`linkedin_connected`	bool	LinkedIn linked	Minor positive — external identity validation.

Key trap: skill_assessment_scores is the ground truth on claimed skills. A candidate who lists 'expert' in Python but scores 22 on the Redrob assessment is lying. This catches keyword stuffers even when their job title looks correct. Always cross-check claimed proficiency against assessment scores for JD-relevant skills.

6. Evaluation Metrics and Scoring

6.1 Composite Score Formula

Your top-100 CSV is scored against a hidden ground truth using this formula:

Metric	Weight	What It Measures
`NDCG@10`	50%	Quality of your top-10 picks. Heavily position-weighted — rank 1 matters far more than rank 10. Getting the best candidates into your first 10 slots dominates your total score.
`NDCG@50`	30%	Quality of your top-50 picks. Less position-sensitive than @10 but still rewards putting good candidates near the top.
`MAP`	15%	Mean Average Precision across all relevance levels. Rewards consistency — finding more relevant candidates anywhere in the list.
`P@10`	5%	Fraction of top-10 that are relevant (tier 3+). Also used as a tiebreaker.

Bottom line: **your top-10 picks are worth 50% of your total score.** Spend the most engineering effort ensuring ranks 1-10 are genuinely the best fits. A near-perfect top-10 with a mediocre 11-100 still beats a mediocre top-10 with a perfect 11-100.

6.2 Relevance Tiers (Ground Truth)

The hidden ground truth grades every candidate into a relevance tier. You never see these, but the metrics are computed against them, so it helps to think in their terms:

Tier	Meaning
`Tier 5`	Best-fit. The 'ideal candidate' profile — applied ML at product companies, shipped retrieval/ranking systems. Some describe this in plain language without buzzwords.
`Tier 3-4`	Relevant. Counts toward P@10. Solid fit with some gaps (e.g. adjacent role, location, or notice-period concerns).
`Tier 1-2`	Weak / adjacent. Some transferable signal but not a real fit.
`Tier 0`	Irrelevant or honeypot. All ~80 honeypots are forced to tier 0.

6.3 Evaluation Pipeline — Five Stages

Stage	What Happens	What Gets You Eliminated
1. Format Validation	Auto-validator runs instantly on every submission	Wrong row count, duplicate ranks, bad candidate_id format, wrong encoding
2. Scoring	Composite computed once after submissions close	Final score below the cutoff for Stage 3 advancement
3. Code Reproduction	Top-N teams' code reproduced in a Docker sandbox (5 min, 16 GB, no GPU, no network); honeypot rate computed	Cannot reproduce within limits; honeypot rate >10% in top-100; missing repo
4. Manual Review	Reasoning quality (6 criteria); git-history authenticity; code quality	Failed reasoning checks; single-commit git history; codebase is just LLM API calls

Stage	What Happens	What Gets You Eliminated
5. Defend-Your-Work	Top-X finalists: 30-min video call to walk through and defend the architecture	Cannot explain architecture; contradicts submitted code; clearly did not build it

6.4 Reasoning Quality — Stage 4 Criteria

Judges sample 10 random rows and check each reasoning entry against these 6 criteria:

- ✓ **Specific facts:** references concrete profile facts (YoE, title, named skills, signal values)
- ✓ **JD connection:** connects to specific JD requirements, not just generic praise
- ✓ **Honest concerns:** where the candidate has gaps, the reasoning acknowledges them
- ✓ **No hallucination:** every claim exists in the actual profile — no invented skills or employers
- ✓ **Variation:** the 10 sampled reasonings are substantively different, not templated
- ✓ **Rank consistency:** tone matches rank (rank-1 enthusiastic; rank-95 measured)

6.5 Tiebreaks & Submission Limit

If two submissions tie on composite score, the order is: higher P@5 wins, then higher P@10, then earlier submission timestamp. You get a maximum of **3 submissions**; your last valid one counts. There is no live leaderboard and no per-submission feedback, so validate locally before every upload.

7. Critical Traps and Disqualifiers

7.1 Honeypot Candidates (~80 profiles)

The dataset contains ~80 honeypot profiles with subtly impossible or internally inconsistent data, forced to relevance tier 0 in the ground truth. If more than 10 honeypots appear in your top-100, your submission is disqualified at Stage 3.

How to identify honeypots:

- **Temporal impossibility:** career_history shows N years at a company founded less than N years ago.
- **Skill duration fraud:** proficiency = 'expert' but duration_months = 0 or 1. Real experts have logged months of use.
- **Assessment contradiction:** claims 'expert' but skill_assessment_scores for that skill is below 25.
- **Experience inflation:** years_of_experience says 12 but the sum of all career_history duration_months is only 24.
- **Impossible timeline:** overlapping jobs that add up to more months than the candidate's stated career span.

7.2 Keyword Stuffers

Candidates with impressive AI skill lists but unrelated job history. The sample submission demonstrates this failure mode (HR Manager at #1). Detection strategies:

- Title-skill mismatch: current_title is 'Marketing Manager' but skills include PyTorch, FAISS, LoRA
- Low skill duration_months: skills at advanced/expert with very few months of actual use
- Low assessment scores: Redrob assessment scores contradict claimed proficiency levels
- Career-description mismatch: no career_history description mentions ML/AI work despite the skills list

7.3 Plain-Language Tier 5s (the inverse trap)

The opposite of a keyword stuffer: a genuinely excellent candidate who describes their work in plain language without buzzwords. Their summary might say 'built the system that decides which profiles recruiters see' rather than 'production RAG pipeline with hybrid retrieval'. A pure keyword matcher misses them; this is exactly why the semantic embedding component (and reading career_history descriptions) matters. Do not require buzzwords to rank someone highly.

7.4 Behavioral Twins

Pairs of candidates that look nearly identical on the static profile (same title, similar skills and experience) but differ sharply on behavioral signals — one is active and responsive, the other inactive with a low response rate. The signals are the tiebreaker. This is the clearest case where the availability multiplier should split two otherwise-equal candidates.

7.5 The Availability Trap

The JD explicitly states: a perfect-on-paper candidate who has not logged in for 6 months with a 5% recruiter response rate is not actually available. Down-weight candidates with:

- last_active_date more than 180 days ago
- recruiter_response_rate below 0.15
- interview_completion_rate below 0.40 (a pattern of ghosting)

- open_to_work_flag = False combined with other low-engagement signals

8. System Architecture — Two-Phase, Six-Layer Design

The solution splits into two clearly separated phases. The split is required by the compute constraint: the ranking step must finish under 5 minutes on CPU, so expensive operations happen offline with no time limit. The ranking step itself is then organised as six progressively narrower layers — each layer takes the output of the previous one and applies a more expensive but more accurate scoring step on a smaller candidate pool.

8.1 Phase 1 — Offline Pre-computation (No Time Limit)

This phase runs once before submission. Outputs are saved to disk and loaded by the ranking script. Pre-computation can take as long as needed; only the ranking step has the 5-minute limit.

Step 1: Load and Parse

Stream all 100,000 candidates from candidates.jsonl. For each, extract profile fields, full career history, skills with duration and proficiency, and all 23 behavioral signals. Store as structured Python dicts.

Step 2: Build TWO Text Representations per Candidate

career_text = all career_history descriptions concatenated, with recency weighting — repeat the most recent role description 2x and weight older roles down to 0.5x. This is what we embed for the main semantic score (career-only, anti-keyword-stuffing). **summary_text** = headline + summary. This gets embedded separately and used ONLY as a consistency check against career_text — a large divergence flags self-marketing inconsistent with actual work history.

Step 3: Embed All Candidates — Career + Summary

Use sentence-transformers (SBERT) with all-MiniLM-L6-v2. Generate two 384-dim embeddings per candidate: career_embeddings.npy (the main one) and summary_embeddings.npy (for consistency). 100K candidates × 2 embeddings × ~1.5ms = ~5 minutes total.

Step 4: Build the BM25 Index on career_text

Use rank_bm25 (pure Python, CPU-fast) over career_text for all 100K candidates. BM25 catches exact technical-term matches that SBERT can miss (rare tokens like 'Qdrant', 'LambdaRank', 'BGE'). Save as bm25_index.pkl.

Step 5: Embed the Job Description and Negative Anchors

Embed the JD text → jd_embedding.npy. Also embed 4-5 explicit anti-fit archetype strings ('Marketing Manager driving brand campaigns', 'HR Manager handling recruitment', 'IT Services consultant on client projects', 'Pure academic researcher with publications'). Save as negative_anchors.npy. These let us subtract anti-fit similarity during ranking.

Step 6: Build FAISS Index on career_embeddings

L2-normalise career_embeddings, then build a FAISS FlatIP (inner product) index. With normalised vectors, inner product equals cosine similarity. Save as faiss_index.bin (~150 MB).

Step 7: Build the Skill Canonicalization Map

Collect every unique skill name across all 100K candidates (~few thousand strings). Embed each one. Embed each JD-required and JD-nice-to-have skill. For each JD skill, find all candidate skills with cosine similarity > 0.75 — these are aliases. Save as skill_canon.pkl. This is how 'Sentence Transformers' auto-matches 'sentence-transformers' and 'vector database' auto-matches FAISS, Qdrant, Pinecone, etc., without you maintaining the list.

Step 8: Pre-compute Structured Features

For each candidate, compute every structured feature: career score sub-components, skill score (using the canonicalization map), experience score, assessment score, availability multiplier, disqualifier flags, all 4 honeypot flags (incl. the new timeline-overlap check), and the trajectory score. Save as features.pkl keyed by candidate_id.

Step 9: Hand-Label the Validation Set and Train the L2 Ranker

Joint team task: label ~200 candidates as tier 0/1/3/5 (see Section 10). Train LightGBM LambdaRank on the labelled set using all sub-scores and raw signal values as features. Save as l2_ranker.pkl.

Step 10: Download and Cache All Models

Save to ./models/: all-MiniLM-L6-v2 (SBERT) and cross-encoder/ms-marco-MiniLM-L-6-v2 (cross-encoder). Network is disabled at runtime — models must load from disk.

8.2 Phase 2 — Six-Layer Ranking Pipeline (under 5 minutes, CPU only)

This is the step reproduced inside the organizer's Docker sandbox. It MUST: complete in under 5 minutes, use only CPU, make zero network calls, and stay under 16 GB RAM. No OpenAI, Anthropic, Gemini, or Hugging Face API calls.

Layer 0: ANN Retrieval — Top 5,000 (~1 second)

Query the FAISS index with the JD embedding. Retrieve the top 5,000 by cosine similarity. We over-retrieve (5K, not 100) so the more expensive downstream layers have material to work with.

Layer 1: Hybrid Lexical + Semantic Re-score — keep top 500 (~5 seconds)

For each of the top 5,000, compute final_semantic = 0.70 × SBERT_cosine + 0.30 × BM25_score. SBERT catches meaning; BM25 catches exact technical-term matches. Keep the top 500 by hybrid score.

Layer 2: Cross-Encoder Re-rank — keep top 200 (~25 seconds)

Use cross-encoder/ms-marco-MiniLM-L-6-v2 on (JD, candidate career_text) pairs. Cross-encoders see both texts together and produce much more accurate relevance scores than bi-encoders. ~50ms per pair × 500 pairs = 25 seconds. Keep the top 200 by cross-encoder score.

Layer 3: Full Composite Score — keep top 150 (~2 seconds)

For each of the 200, compute the full composite using pre-computed structured features (see Section 9): weighted components + trajectory + consistency check - negative anchor penalty, then multiplied by availability, disqualifier, and honeypot multipliers. Sort descending; keep top 150.

Layer 4: LightGBM LambdaRank L2 Re-rank — top 100 (<1 second)

Run the trained LightGBM model on the top 150. Features = all sub-scores + raw behavioral signal values. The L2 model learned the optimal feature combination from the validation set — much better than our hand-weighted formula at the top of the list. Output is the final top 100.

Layer 5: Assessment Hard Gate for Top 20 (<1 second)

Second pass on ranks 1-20: if a candidate claims 'expert' in a JD-required skill but their skill_assessment_scores for that skill is below 40, demote them out of the top 20. The composite score with 15% assessment weight is not enough to stop a confident liar from sitting at rank 4 — this gate is a defensive line for the highest-value ranks.

Layer 6: Final Honeypot Safety Check (<1 second)

Verify zero honeypots remain in the top 100. If any survived the multiplier penalty, replace them with candidates from ranks 101-200.

Output: Reasoning Generation + CSV

For each of the top 100, build a 1-2 sentence reasoning string from real candidate facts using varied templates (no LLM, no repetition). Write team_xxx.csv (candidate_id, rank, score, reasoning), then run validate_submission.py to confirm format compliance.

> **Total ranking-step time estimate:** ~70 seconds (load 30s + layers ~35s + reasoning & output ~5s). Well within the 5-minute budget.

8.3 Architecture Flow Diagram

```
OFFLINE PHASE (no time limit - run once before submission)

  candidates.jsonl
     -> [Parser + Feature Extractor] -> features.pkl
     -> [career_text builder (recency-wgtd)]
          -> [SBERT MiniLM-L6] -> career_embeddings.npy
          -> [BM25 Indexer] -> bm25_index.pkl
     -> [summary_text builder] -> [SBERT] -> summary_embeddings.npy
     -> [Unique-skill Embedder] -> skill_canon.pkl

  job_description.docx
     -> [JD Text Extractor] -> [SBERT] -> jd_embedding.npy
     -> [Anti-fit Archetypes] -> [SBERT] -> negative_anchors.npy

  validation_labels.csv (hand-labelled tier 0/1/3/5)
     -> [LightGBM LambdaRank Trainer] -> l2_ranker.pkl

  career_embeddings.npy
     -> [L2 Normalise] -> [FAISS IndexBuilder] -> faiss_index.bin


RANKING PHASE (<= 5 min, CPU only, no network)

  L0 FAISS ANN Search
        query JD embedding -> retrieve top 5,000 by cosine

  L1 Hybrid Lexical-Semantic Re-score
        score = 0.70 * SBERT_cos + 0.30 * BM25
        -> keep top 500

  L2 Cross-Encoder Re-rank (ms-marco-MiniLM-L-6-v2)
        score (JD, career_text) pairs
        -> keep top 200

  L3 Full Composite Score
        0.25*sem + 0.30*career + 0.20*skill
         + 0.10*exp + 0.15*assess
         + trajectory_adj + consistency_adj
         - negative_anchor_penalty
        * availability * disqualifier * honeypot
        -> keep top 150

  L4 LightGBM LambdaRank L2 Re-rank
        features = all sub-scores + raw signal values
        -> top 100

  L5 Assessment Hard Gate (ranks 1-20 only)
```

```
      expert claim + assessment < 40 -> demote

L6 Final Honeypot Safety Check
      any honeypot in top 100 -> replace from 101-200

-> Reasoning Generator -> submission.csv
```

9. Detailed Scoring Logic

The composite score is a weighted combination of five component scores, plus three small additive adjustments (trajectory, consistency, negative-anchor penalty), all multiplied by a behavioral availability modifier and two hard-penalty multipliers. Component scores are normalised to [0.0, 1.0]; adjustments are bounded.

9.1 Master Formula

```
base_score =
  (
      0.25 * semantic_score # hybrid SBERT+BM25 on career text
    + 0.30 * career_score # company type, role, progression
    + 0.20 * skill_score # skill match x duration x proficiency
    + 0.10 * experience_score # YoE fit, location, education
    + 0.15 * assessment_score # Redrob assessments on JD skills
  )
  + trajectory_adjustment # ±0.05 career direction
  + consistency_adjustment # ±0.03 summary vs career-text
  - negative_anchor_penalty # 0 to 0.10 anti-fit similarity

base_score = clamp(base_score, 0.0, 1.0)

composite_score =
  base_score
  * availability_multiplier # behavioral modifier (0.10 to 1.25)
  * disqualifier_multiplier # 0.05 if disqualified, else 1.0
  * honeypot_penalty # 0.0 if honeypot, else 1.0
```

Why these weights: career descriptions carry the strongest signal of actual fit (the JD literally says so), so career_score gets the largest share (0.30). Assessment scores cannot be self-reported or stuffed, so they get more weight than experience (0.15 vs 0.10). Experience (YoE, location, education) is mostly a coarse filter and gets the smallest weight.

9.2 Component 1: Semantic Score — Hybrid SBERT + BM25 (weight 0.25)

Two key changes from a naive embedding approach:

- **Career text only, not summary or headline.** Summaries are self-written marketing copy — anyone can stuff them with buzzwords. Career-history descriptions are tied to specific job titles and durations, which are much harder to fake. We embed career_text only.
- **Recency-weighted.** When building career_text, repeat the most recent role description 2x and weight older roles down to 0.5x. The embedding represents 'who this person is now', not an average of their entire career.
- **Hybrid SBERT + BM25.** SBERT catches meaning; BM25 catches exact technical-term matches (rare tokens like 'Qdrant', 'BGE', 'LambdaRank' that SBERT can miss).

```
final_semantic = 0.70 * SBERT_cosine + 0.30 * BM25_normalised
# both computed on career_text only
```

9.3 Summary Consistency Adjustment (±0.03)

The summary embedding is NOT used as a positive signal — it is used as an anti-fraud check. Compute cosine similarity between summary_embedding and career_embedding. If they align well, the candidate's self-description matches their actual work history (small + adjustment). If they diverge sharply, the summary is likely stuffed with buzzwords inconsistent with the work (small – adjustment).

```
sim = cosine(summary_embedding, career_embedding)
  sim > 0.70 -> +0.03
```

```
0.50 <= sim <= 0.70 -> 0.00
sim < 0.50 -> -0.03
```

9.4 Component 2: Career Score (weight 0.30)

Five sub-components, weighted and combined:

- **Company type (40%):** product-company experience scores highest. Startups/scale-ups/product companies = 1.0; mixed career = 0.5; pure IT services (TCS, Infosys, Wipro, Accenture) = 0.1.
- **Role relevance (30%):** ML/AI Engineer, Research Engineer, applied Data Scientist = 1.0; Backend Engineer with ML exposure = 0.5; Marketing/HR/Consulting = 0.05.
- **Production deployment signal (10%):** binary. Description contains 'deployed', 'production', 'serving', 'real users', 'A/B test', or 'rollout' = 1.0, else 0.0.
- **Relevant domain indicator (10%):** binary. Description mentions 'ranking', 'retrieval', 'search', 'recommendation', 'vector', 'embedding', 'NDCG', 'index', or 'semantic' = 1.0, else 0.0.
- **Tenure stability (10%):** 1.0 - (roles with duration_months < 18) / total_roles.

9.5 Component 3: Skill Score with Canonicalization (weight 0.20)

Skill matching uses embedding-based canonicalization: we embed every unique skill string in the dataset, embed each JD skill, and treat any candidate skill with cosine similarity > 0.75 to a JD skill as a match. This auto-expands 'vector databases' to FAISS / Qdrant / Pinecone / Milvus and matches 'Sentence Transformers' against 'sentence-transformers' without you maintaining the alias list manually.

```
# For each matched JD-relevant skill in the candidate's skill list:
per_skill_weight = base_weight * proficiency_mult * duration_mult

base_weight:
  JD Required (embeddings, vector DB, Python, eval) = 1.0
  JD Nice-to-have (LoRA, LTR, NLP, recommendation) = 0.4

proficiency_mult:
  expert = 1.0 advanced = 0.8 intermediate = 0.5 beginner = 0.2

duration_mult (months of actual use):
  >= 48 mo = 1.0 # credible expert
  24-48 mo = 0.8
  12-24 mo = 0.6
   6-12 mo = 0.4
   < 6 mo = 0.1 # probable keyword stuffer

skill_score = sum(per_skill_weights) / max_possible_sum # -> [0,1]
```

9.6 Component 4: Experience Score (weight 0.10)

- **YoE fit (40%):** 5-9 yrs = 1.0; 4 or 10 yrs = 0.7; 3 or 11-13 yrs = 0.4; outside = 0.2.
- **Location fit (35%):** Pune/Noida = 1.0; other Tier-1 Indian city = 0.8; other India + willing_to_relocate = 0.65; other India without relocation = 0.3; outside India = 0.2.
- **Education tier (25%):** tier_1 = 1.0, tier_2 = 0.8, tier_3 = 0.6, tier_4/unknown = 0.4; a CS/ECE/Math/Stats field adds a 0.1 bonus.

9.7 Component 5: Assessment Score (weight 0.15)

Average of skill_assessment_scores for skills in the JD required list — the ground-truth verification of claimed skills. If a candidate has no assessment for a JD skill, that skill contributes 0. Normalised to [0,1] by dividing by 100. Weighted higher than experience because assessment scores cannot be faked.

9.8 Trajectory Score (±0.05)

Two candidates with the same current title aren't equal if one went Sales → Data → ML and the other went ML → Architecture → Consulting. We score career direction by comparing the last 2 roles' ML-alignment against the older roles:

```
for each role, compute title_alignment = cosine(title_embedding, JD_title_embedding)

recent_avg = mean(title_alignment of last 2 roles)
older_avg = mean(title_alignment of all earlier roles)

delta = recent_avg - older_avg

  delta > 0.10 -> +0.05 # moving toward ML
  -0.10 to 0.10 -> 0.00 # flat
  delta < -0.10 -> -0.05 # moving away from ML
```

9.9 Negative Anchor Penalty (0 to 0.10)

We embed 4-5 explicit anti-fit archetype strings during pre-computation (Marketing Manager, HR Manager, IT Services Consultant, Pure Academic Researcher). At ranking time, compute the candidate's career_embedding cosine similarity to each anchor and subtract the maximum from the base score. This catches keyword stuffers and anti-fits at the embedding level, complementing the structured disqualifier check.

```
max_anti_sim = max(cosine(career_embedding, anchor) for anchor in anchors)

  max_anti_sim < 0.40 -> penalty = 0.00
  0.40 <= max < 0.55 -> penalty = 0.05
  max_anti_sim >= 0.55 -> penalty = 0.10
```

9.10 Availability Multiplier (Behavioral Signals)

```
# Start at 1.0, apply additive adjustments, then clamp.

days_inactive = (today - last_active_date).days
  < 14 days -> +0.15
  14-30 days -> +0.10
  30-90 days -> 0.00
  90-180 days -> -0.10
  > 180 days -> -0.25

recruiter_response_rate
  > 0.70 -> +0.10
  0.40-0.70 -> 0.00
  0.20-0.40 -> -0.05
  < 0.20 -> -0.15

engagement bonuses (additive):
  open_to_work_flag = True -> +0.05
  applications_submitted_30d > 3 -> +0.05
  github_activity_score > 60 -> +0.05
  saved_by_recruiters_30d > 5 -> +0.05

notice_period_days
  <= 30 -> 0.00 30-60 -> -0.03 60-90 -> -0.07 > 90 -> -0.12

availability_multiplier = clamp(1.0 + sum(adjustments), 0.10, 1.25)
```

9.11 Disqualifier Multiplier

Set to 0.05 (effectively removing from top-100) if the candidate matches any of:

- All career_history companies are known IT-services firms (TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini, HCL, Tech Mahindra)

- Title-chaser pattern: more than 60% of roles lasted under 18 months AND titles show escalation
- Wrong primary specialisation: all skills/descriptions are CV/Speech/Robotics with zero NLP or retrieval signal

9.12 Honeypot Detection — 4 Checks

Honeypot detection now includes a fourth check for physically impossible career timelines:

```
def is_honeypot(candidate) -> bool:

    # Check 1: expert skill with near-zero usage
    for skill in candidate['skills']:
        if skill['proficiency'] == 'expert' and skill.get('duration_months', 0) < 3:
            return True

    # Check 2: stated YoE vs summed career history
    total = sum(r['duration_months'] for r in candidate['career_history'])
    stated = candidate['profile']['years_of_experience'] * 12
    if total < stated * 0.35: # allow for gaps
        return True

    # Check 3: assessment contradicts an expert claim
    scores = candidate['redrob_signals'].get('skill_assessment_scores', {})
    for skill in candidate['skills']:
        n = skill['name']
        if n in scores and skill['proficiency'] == 'expert' and scores[n] < 25:
            return True

    # Check 4 (NEW): physically impossible overlapping roles
    # Two full-time roles overlapping by > 6 months is impossible.
    roles = sorted(candidate['career_history'], key=lambda r: r['start_date'])
    for i in range(len(roles) - 1):
        for j in range(i + 1, len(roles)):
            overlap = compute_overlap_months(roles[i], roles[j])
            if overlap > 6:
                return True

    return False
```

9.13 Assessment Hard Gate for Top 20

After all scoring and L2 re-ranking, a second pass checks ranks 1-20 for assessment fraud. Composite scoring with 15% assessment weight is NOT enough to stop a confident liar from sitting at rank 4. The hard gate is a defensive line for the highest-value positions, where NDCG@10 weighting is most punishing:

```
for rank, cand in enumerate(top_100[:20], 1):
    scores = cand['redrob_signals'].get('skill_assessment_scores', {})
    for skill in cand['skills']:
        if skill['name'] in JD_REQUIRED_SKILLS:
            if skill['proficiency'] == 'expert':
                if scores.get(skill['name'], 100) < 40:
                    # Demote: move this candidate out of top 20
                    demote(cand, new_rank_floor=21)
                    break
```

10. Validation Set and Learning-to-Rank Layer

Hand-weighting the composite formula in Section 9 gives you educated guesses. To actually win, we need ground truth — a held-out validation set — and a model that learns the optimal feature combination from it. This section adds both: the labeling protocol and the LightGBM LambdaRank layer that re-ranks the top 150 just before submission.

10.1 Why We Need a Validation Set

We never see the organizers' ground truth, but we still need *some* signal to tell us whether a weight change improves the ranking. Without a validation set, every weight tweak is a coin toss and every ablation is theatre. A small hand-labelled set is the difference between guessing and engineering.

10.2 Hand-Labeling Protocol — Day 1 (All 4 Team Members)

- Take all 50 candidates from sample_candidates.json plus 150 random pulls from candidates.jsonl. Total: ~200 candidates.
- All 4 team members label each candidate independently as tier 0 / 1 / 3 / 5 using the criteria below.
- For any candidate where labels disagree by more than 1 tier, a 2-minute team discussion resolves to consensus.
- Save as validation_labels.csv with columns: candidate_id, tier.
- Target time: 3-4 hours all together. This is the highest-leverage 3 hours of the entire hackathon.

10.3 Tier Criteria for Labeling

Tier	Criteria
`Tier 5`	Best fit. 5-9 YoE applied ML at a product company. Career descriptions explicitly mention shipping retrieval/ranking/recommendation systems. Pune/Noida or willing to relocate. Active on platform (last 30d, response rate > 0.5).
`Tier 3`	Relevant with gaps. Solid ML/AI background and product-company experience but one or two real concerns (longer notice, slightly off location, adjacent role title, mid response rate).
`Tier 1`	Weak / adjacent. Some transferable signal — maybe a Backend Engineer who used ML once, or a Research Scientist without production work. Not a real fit.
`Tier 0`	Irrelevant or honeypot. Wrong role entirely, OR matches one of our 4 honeypot checks.

10.4 Ablation Harness

Once labels exist, every weight change can be measured. Build a tiny harness that scores the validation set with the current pipeline, computes NDCG@10 / NDCG@50 / MAP / P@10 against the labels, and reports the deltas. Run it before AND after every change. Components that don't move the metrics are dead weight — drop them.

```
# ablation.py — toggle one component at a time
for component in ['semantic', 'career', 'skill', 'experience',
                  'assessment', 'trajectory', 'consistency',
                  'negative_anchor', 'availability', 'cross_encoder']:
    scores = rank_with_component_disabled(component, validation_set)
    ndcg10 = compute_ndcg_at_k(scores, labels, k=10)
    print(f'{component:20s} drop_NDCG@10 = {baseline - ndcg10:+.4f}')
```

10.5 LightGBM LambdaRank — The L2 Re-ranker

Even with tuned weights, hand-coded linear combination has limits. LightGBM LambdaRank learns feature interactions automatically. Trained on the validation set, it re-ranks the top 150 from the composite scorer into the final top 100. CPU-only, training takes seconds, inference is microseconds.

Feature set fed to LightGBM (per candidate):

- All 5 component scores (semantic, career, skill, experience, assessment)
- All career sub-components individually (company_type, role_relevance, production_signal, domain_indicator, tenure_stability)
- Trajectory adjustment, consistency adjustment, negative anchor penalty (raw, before clamping)
- Cross-encoder score from Layer 2
- Raw behavioral signal values (response_rate, days_inactive, notice_period, github_score, interview_completion_rate, etc.)
- Honeypot check raw values (overlap_months, yoe_ratio, expert_zero_count)

```
import lightgbm as lgb

# Training (offline, once)
train_data = lgb.Dataset(X_train, label=y_train, group=[len(y_train)])
params = {
    'objective': 'lambdarank',
    'metric': 'ndcg',
    'ndcg_eval_at': [10, 50],
    'learning_rate': 0.05,
    'num_leaves': 31,
    'max_depth': 6,
}
model = lgb.train(params, train_data, num_boost_round=200)
model.save_model('precomputed/l2_ranker.pkl')

# Inference (in ranking step, Layer 4)
scores = model.predict(X_top150)
top_100 = top_150[np.argsort(-scores)][:100]
```

10.6 Guarding Against Overfitting

With only ~200 labels, LightGBM can easily overfit. Three guards: (1) 5-fold cross-validation during training to pick num_boost_round; (2) regularization (max_depth = 6, num_leaves = 31, lambda_l2 = 1.0); (3) compare the L2-re-ranked top-100 against the composite top-100 on the validation set — if L2 hurts NDCG, fall back to the composite ranking.

11. The Three MVPs — Our Differentiators

When you defend this work in the Stage 5 interview, these are the three points to lead with. Each one targets a failure mode that most other teams will fall into. Together they are the reason this solution can compete for the top of the leaderboard.

MVP 1: Career-Description-Only Embedding + BM25 Hybrid

What most teams will do: embed the candidate's full profile (headline + summary + experience + skills) with SBERT and rank by cosine similarity to the JD embedding.

What we do: embed career_history descriptions only (with recency weighting), then combine with BM25 in a 0.70 / 0.30 blend. Summaries are used only as an anti-fraud consistency check, not a positive signal.

Why it matters: summaries are self-written marketing copy — trivial to stuff with buzzwords. Career descriptions are tied to specific titles and durations, which are much harder to fabricate. The HR Manager at #1 in the sample submission has 9 AI keywords in their skills list but a career history of HR work — embedding career text only kills that ranking outright. BM25 catches exact technical-term matches that SBERT can miss (rare tokens like 'Qdrant', 'BGE', 'LambdaRank'). This is the single biggest defence against keyword stuffing.

MVP 2: Behavioral Signal Multiplier + Assessment Hard Gate

What most teams will do: rank by profile fit alone, possibly with behavioral signals as a small additive bonus.

What we do: behavioral signals act as a *multiplicative* modifier on the base score (0.10× to 1.25× range), then a hard gate enforces assessment integrity in the highest-value positions (ranks 1-20). Claimed-expert + assessment-below-40 demotes the candidate out of the top 20 regardless of every other score.

Why it matters: the JD explicitly states that a perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% response rate is not actually hirable. Our multiplier directly down-weights them. The hard gate then protects ranks 1-20 from confident liars — composite scoring with 15% assessment weight is NOT enough to stop someone from sitting at rank 4 while their Python assessment score is 22. NDCG@10 is 50% of our total score; this is where we defend it.

MVP 3: Multi-Layer Honeypot Detection (4 Checks)

What most teams will do: miss the honeypots entirely and lose at Stage 3 (the 10% honeypot-in-top-100 disqualifier), or implement one or two checks and get most but not all.

What we do: four independent cross-validation checks: skill duration fraud, YoE vs career-history mismatch, assessment-score contradiction, and physically-impossible overlapping roles (timeline-overlap > 6 months). Any single check flags as honeypot.

Why it matters: approximately 80 honeypots exist in the 100K pool. Naive ranking puts many of them near the top because their skills lists look great. With four checks instead of one, we have multiple independent ways to catch each honeypot — even a candidate engineered to pass three of our checks usually fails the fourth. This is what survives Stage 3 reproduction.

Defensibility note: in the Stage 5 interview, the questions will probe these three areas — 'why didn't you embed the summary?', 'how do you handle inactive candidates?', 'how do you know your top 100 has no honeypots?'. Be ready to walk through each MVP with the specific numbers from your run (e.g. 'in our top 100, the assessment hard gate demoted 3 candidates from the top 20 — we can show you which and why').

12. Tech Stack and Libraries

12.1 Core Libraries

Library	Version	Purpose	Install
`sentence-transformers`	2.7+	SBERT (all-MiniLM-L6-v2) for career + summary embeddings; cross-encoder for L2 re-rank.	`pip install sentence-transformers`
`rank-bm25`	0.2+	BM25 retrieval over career_text. Pure Python, CPU-fast.	`pip install rank-bm25`
`faiss-cpu`	1.8+	FAISS FlatIP index. 100K vectors searched in <1s on CPU.	`pip install faiss-cpu`
`lightgbm`	4.3+	LambdaRank L2 re-ranker on top 150. Trained on validation labels.	`pip install lightgbm`
`numpy`	1.26+	Vector ops, .npy storage, L2 normalisation.	`pip install numpy`
`pandas`	2.2+	CSV output, structured data manipulation.	`pip install pandas`
`scikit-learn`	1.4+	MinMaxScaler; NDCG/MAP metric implementations for ablation.	`pip install scikit-learn`
`tqdm`	4.66+	Progress bars during the 100K embedding loop.	`pip install tqdm`
`python-dateutil`	2.9+	Date parsing for last_active_date and career dates.	`pip install python-dateutil`
`pyyaml`	6.0+	Reading/writing submission_metadata.yaml.	`pip install pyyaml`

12.2 Models We Use

Model	Purpose	Size	CPU Speed	Where Used
`all-MiniLM-L6-v2`	Bi-encoder for SBERT embeddings	~22 MB	~1.5 ms/cand	Offline embedding for career + summary + skills + JD + anchors
`ms-marco-MiniLM-L-6-v2`	Cross-encoder for re-ranking	~80 MB	~50 ms/pair	Layer 2: re-rank top 500 with (JD, career_text) pairs
`LightGBM LambdaRank`	Learned L2 re-ranker	~5 MB	<1 ms/cand	Layer 4: re-rank top 150 using all sub-scores as features

Both transformer models must be cached to ./models/ during offline pre-computation — network is disabled at runtime. LightGBM is trained offline once on the validation labels.

12.3 Project Directory Structure

```
redrob-ranker/
  data/
    candidates.jsonl # 465 MB full dataset
    job_description.docx
    validation_labels.csv # hand-labelled ~200 candidates
```

```
models/
  all-MiniLM-L6-v2/ # SBERT cached locally
  ms-marco-MiniLM-L-6-v2/ # cross-encoder cached locally
precomputed/
  career_embeddings.npy # 100K x 384 float32
  summary_embeddings.npy # 100K x 384 float32
  jd_embedding.npy
  negative_anchors.npy # 4-5 anti-fit archetype vectors
  faiss_index.bin # FAISS FlatIP on career_embeddings
  bm25_index.pkl # BM25 over career_text
  skill_canon.pkl # JD skill -> aliases via embeddings
  features.pkl # all structured features, 100K
  l2_ranker.pkl # LightGBM LambdaRank model
src/
  build_features.py # offline: parse + extract features
  build_embeddings.py # offline: SBERT + BM25 + skill canon
  build_index.py # offline: FAISS index
  train_l2.py # offline: train LightGBM on labels
  ablation.py # offline: validation-set ablations
  rank.py # RANKING STEP (six layers, main entry)
  scorer.py # all component scores + multipliers
  reasoning.py # rule-based reasoning generator
  honeypot.py # 4-check honeypot detection
validate_submission.py # provided by organizers
requirements.txt
README.md
submission_metadata.yaml
```

13. Compute Constraints and Compliance

Constraint	Limit	Our Approach and Estimate
Total runtime (ranking step)	<= 5 min wall-clock	Load ~30s + L0 FAISS <1s + L1 hybrid ~5s + L2 cross-encoder ~25s + L3 composite ~2s + L4 LightGBM <1s + L5/L6 gates <1s + output ~5s. Total ~70 seconds.
Memory	<= 16 GB RAM	career_embeddings + summary_embeddings ~300 MB; features.pkl ~250 MB; FAISS index ~150 MB; BM25 index ~200 MB; LightGBM model ~5 MB; both models in memory ~120 MB. Total ~1.0 GB, well under 16 GB.
Compute	CPU only — no GPU	SBERT MiniLM and cross-encoder MiniLM both run on CPU; faiss-cpu is a native CPU build; LightGBM CPU mode; rank-bm25 is pure Python. No CUDA required.
Network	No external API calls	Zero API calls during ranking. Both models, BM25 index, FAISS index, and LightGBM all loaded from disk. No OpenAI/Anthropic/Cohere/Gemini/HuggingFace API.
Disk	<= 5 GB intermediate	Embeddings (career + summary + JD + anchors) ~300 MB + FAISS ~150 MB + BM25 ~200 MB + features ~250 MB + models ~100 MB + LightGBM ~5 MB = ~1 GB total. Well within 5 GB.

Critical: both transformer models must be downloaded and saved to disk DURING the offline phase — the ranking step cannot download them at runtime because network is disabled. In build_embeddings.py: SentenceTransformer('all-MiniLM-L6-v2', cache_folder='./models/') and CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2', cache_folder='./models/'). In rank.py: load from the local paths. Same for the LightGBM ranker — train offline once, lgb.Booster(model_file='./precomputed/l2_ranker.pkl') at runtime.

14. Submission Format and Requirements

14.1 Required CSV Format

```
candidate_id,rank,score,reasoning
CAND_0042871,1,0.987,"Senior AI Engineer, 7yr building retrieval
  systems at product companies; FAISS+Qdrant in prod; Pune-based."
CAND_0019884,2,0.973,"6yr applied ML at a Series B startup; shipped
  vector search at scale; 30d notice and actively applying."
CAND_0091235,3,0.962,"Strong NLP + retrieval with Elasticsearch in
  prod; concern: 120d notice and last active 45 days ago."
...
CAND_0007729,100,0.412,"Adjacent skills only; data-engineering
  background; included at the boundary given strong engagement."
```

Columns must be exactly candidate_id, rank, score, reasoning in that order. Note how reasoning tone tracks rank — confident at the top, candid about concerns lower down.

14.2 Pre-Submission Checklist

- [ ] Exactly 100 data rows plus 1 header row (101 total)
- [ ] Ranks 1 through 100 each appear exactly once
- [ ] Each candidate_id appears exactly once and exists in candidates.jsonl

- [ ] Scores are non-increasing as rank increases
- [ ] File is UTF-8 with a .csv extension, named as your participant ID (e.g. team_xxx.csv)
- [ ] Equal scores are tie-broken by candidate_id ascending
- [ ] validate_submission.py passes with no errors

14.3 Three-Submission Limit

You get exactly 3 submissions; your last valid one counts. There is no live leaderboard and no per-submission feedback. Validate locally first and do not waste a submission on a format error.

15. Team Role Division

With 4 members, here is the recommended split. Interfaces are clean so all four can work in parallel after the joint setup and validation-labeling sessions.

Person	Role	Primary Deliverables	Key Files
Person 1	Data + Features + Honeypots	candidates.jsonl parser; structured feature extraction for all 100K (career sub-scores, skill canonicalization map, trajectory, all 4 honeypot checks incl. timeline overlap); features.pkl + skill_canon.pkl outputs.	`build_features.py, honeypot.py, build_skill_canon.py`
Person 2	Embeddings + Retrieval	Career-text builder with recency weighting; SBERT pipeline for career + summary + JD + negative anchors; BM25 index over career_text; FAISS index; cross-encoder integration for Layer 2.	`build_embeddings.py, build_bm25.py, build_index.py`
Person 3	Scoring + L2 Ranker	Hybrid semantic score; all 5 component scores; trajectory, consistency, and negative-anchor adjustments; availability + disqualifier multipliers; assessment hard gate; LightGBM L2 training; rank.py main entry point.	`scorer.py, rank.py, train_l2.py`
Person 4	Validation + Reasoning + DevOps	Validation labeling protocol coordination; ablation harness with NDCG/MAP/P@K implementations; rule-based reasoning generator with varied templates; CSV output; GitHub repo + README; sandbox deployment.	`ablation.py, reasoning.py, README.md, sandbox`

Two joint sessions before parallel work begins:
(1) Setup, 2-3 hours. Agree on the parsed-candidate data structure, the features.pkl schema, and the interface between scorer.py and rank.py.
(2) Validation labeling, 3-4 hours. All four members label ~200 candidates as tier 0/1/3/5. This is the single highest-leverage block of time in the entire hackathon — without it, the L2 ranker has nothing to train on and every weight is a guess.

16. Implementation Roadmap

Phase	Tasks	Est. Time
0. Joint Setup	Clone repo structure; install all dependencies (sentence-transformers, faiss-cpu, rank-bm25, lightgbm, etc.); test-parse sample_candidates.json; agree on data structures and module interfaces.	2-3 hrs
1. Validation Labeling (joint)	All 4 members label ~200 candidates as tier 0/1/3/5; resolve disagreements by discussion; save validation_labels.csv. Highest-leverage block of the hackathon.	3-4 hrs
2. Data Pipeline	Build the parser; extract structured features for 50 samples; implement all 4 honeypot checks (incl. timeline overlap); build skill canonicalization map; output features.pkl + skill_canon.pkl.	4-5 hrs
3. Embeddings + Retrieval	Build career_text (recency-weighted) and summary_text; embed both with SBERT for 50 samples first, sanity-check, then run full 100K (~5 min CPU); build BM25 index; embed JD + negative anchors; build FAISS index.	5-7 hrs

Phase	Tasks	Est. Time
4. Scoring Engine	Implement hybrid semantic score (SBERT + BM25); all 5 components; trajectory, consistency, negative-anchor adjustments; availability and disqualifier multipliers; assessment hard gate. Verify top-5 on samples make sense against the JD.	5-6 hrs
5. Cross-Encoder + L1 Pipeline	Integrate cross-encoder for Layer 2; wire L0 (FAISS) -> L1 (hybrid) -> L2 (cross-encoder) -> L3 (composite). End-to-end run on 100K; time the ranking step (must be < 5 min); check honeypot rate in top 100.	3-4 hrs
6. Ablation + L2 Training	Run ablation harness on validation set; measure NDCG@10/50, MAP, P@10 with each component disabled; drop dead-weight components; train LightGBM LambdaRank on validation labels; cross-validate; integrate as Layer 4.	4-5 hrs
7. Reasoning + Output	Build varied reasoning templates; verify real facts only (no hallucinations); tone matches rank position; CSV output with strict format compliance.	2-3 hrs
8. Validation + Sandbox	Run validate_submission.py; fix any errors; deploy sandbox to HuggingFace Spaces or Streamlit; end-to-end test on sample.	2-3 hrs
9. Final Submission	Review the top-10 manually (defend each one in 1 sentence); confirm real git history with multiple commits; submit CSV; fill in all portal metadata.	1 hr

Total estimated effort: ~32-42 person-hours, spread across 4 people = roughly 8-11 hours per person of focused work, plus ~7 hours of joint sessions (setup + labeling). Plan for a buffer of at least 4-6 hours for unexpected debugging, especially around the cross-encoder timing and LightGBM overfitting.

17. Sandbox and GitHub Requirements

17.1 GitHub Repository — Required Contents

- A clear README.md with project description, setup instructions, and a single command to reproduce the CSV
- Full source code — no hidden steps, no manual edits outside the repo
- requirements.txt with all dependencies and versions pinned (sentence-transformers, faiss-cpu, rank-bm25, lightgbm, etc.)
- submission_metadata.yaml at the repo root (filled from the organizer template)
- Pre-computed artifacts (career/summary embeddings, FAISS index, BM25 index, features.pkl, l2_ranker.pkl) OR scripts that generate them
- Cached models in ./models/ (SBERT and cross-encoder) so the ranking step can run offline
- A documented single reproduce command, e.g. python rank.py --candidates ./candidates.jsonl --out ./submission.csv
- Real git history with multiple commits showing iteration — NOT a single final-code dump

17.2 Sandbox Link — Required Demo

A hosted environment where organizers verify the ranker runs. It must accept a small candidate sample (up to 100), run the pipeline, and produce a ranked CSV. It does NOT need to handle the full 100K pool.

- **HuggingFace Spaces (Streamlit)** — free tier, easy to deploy. RECOMMENDED.
- **Streamlit Cloud** — free tier, auto-deploys from GitHub.
- **Google Colab** — a shareable end-to-end notebook. Less preferred (manual run).
- **Replit** — a free public repl. Good option.
- **Docker** — a Dockerfile + docker pull command in the README. Most robust for Stage 3.

Quick Reference Card

Dataset: 100,000 candidates (465 MB) | Job: Senior AI Engineer, Redrob AI, Pune/Noida
Output: top-100 ranked CSV | Score: 50% NDCG@10 + 30% NDCG@50 + 15% MAP + 5% P@10
Compute: <=5 min, CPU only, no network, <=16 GB RAM | Submissions: 3 max, last counts
Disqualified if honeypot rate >10% in top-100 | 4 honeypot checks: skill duration, YoE mismatch, assessment contradiction, timeline overlap
Pipeline: FAISS top-5K -> hybrid SBERT+BM25 -> cross-encoder -> composite -> LightGBM L2 -> assessment gate
Weights: 0.25 semantic + 0.30 career + 0.20 skill + 0.10 exp + 0.15 assess
Plus: trajectory ±0.05, consistency ±0.03, negative anchor up to –0.10
Models: all-MiniLM-L6-v2 (bi) + ms-marco-MiniLM-L-6-v2 (cross) + LightGBM LambdaRank
MVPs: career-only embed + BM25 | behavioral multiplier + assessment gate | 4-check honeypot